

Devoir surveillé

Durée 1h30
(Corrections)

Exercice 1 (La phrase cachée - 8 pts)

Un fichier binaire contenant des données sensibles a été intercepté. Nos experts ont réussi à comprendre comme le déchiffrer et font appel à vous pour coder un programme permettant de retrouver la phrase cachée.

Une position est stockée au début du fichier (un `off_t`). Il faut se déplacer à cette position qui comprend une position gauche (un `off_t`), un caractère (un `char`) et une position droite (un `off_t`), soit au total 17 octets. Une des deux positions peut être nulle, voire les deux ; dans ce dernier cas, il s'agit d'un cul-de-sac. Pour trouver la phrase cachée, il faut explorer tous les chemins possibles. La phrase termine forcément par un point, dans un cul-de-sac.

Votre programme prend en arguments le nom du fichier à déchiffrer (le fichier d'entrée) et le nom du fichier qui contiendra la phrase cachée (le fichier de sortie).

1°) Donnez la portion de code permettant au programme d'ouvrir le fichier de sortie, tout en vérifiant qu'il n'existe pas déjà. **Vous devez gérer les erreurs dans cette question.**

Solution : (2 pts)

```
int fd;

if((fd = open(argv[1], O_WRONLY | O_CREAT | O_EXCL, S_IRUSR | S_IWUSR)) == -1) {
    if(errno == EEXIST) {
        fprintf(stderr, "Le_fichier_'%s'_existe_déjà.\n", argv[1]);
    }
    else {
        fprintf(stderr, "Erreur_lors_de_l'ouverture_du_fichier_'%s'_", argv[1]);
        perror("");
    }
    exit(EXIT_FAILURE);
}
```

2°) Nous souhaitons utiliser une pile pour parcourir le fichier. Expliquez ce qu'elle contient et comment l'utiliser.

Solution : (1 pt) La pile contient des éléments qui correspondent à une position (un `off_t`) et une chaîne de caractères (un `char[]`). Lorsqu'on arrive à une position du fichier, on empile les positions de gauche et de droite, si elles ne sont pas nulles. Elles sont associées avec les caractères déjà trouvés pour atteindre la position actuelle en y ajoutant le caractère actuel. Le programme s'arrête lorsque la phrase est trouvée (positions droite et gauche nulles et le caractère égal à '.') ou lorsque la pile est vide (cas d'erreur).

3°) Est-il possible d'utiliser une structure pour lire les positions et le caractère en un seul appel? Si oui, donnez la structure en C, sinon expliquez pourquoi.

Solution : (1 pt) Oui, si on utilise un tableau de 17 octets et que l'on utilise ensuite les adresses du tableau pour retrouver les positions et le caractère. Non, si on parle d'une structure contenant un `off_t`, un `char` et un `off_t`. En effet, cela impliquerait des octets de bourrage et nous serions à plus de 17 octets.

4°) Comment peut-on détecter qu'une position lue est invalide? Précisez les appels systèmes nécessaires.

Solution : (1 pt) Les positions doivent être comprises entre 0 et la taille du fichier. Cette dernière peut être récupérée à l'aide de `lseek(fd, 0, SEEK_END)` où `fd` est le descripteur du fichier d'entrée. Lorsqu'on lit une position avec `read`, on vérifie qu'elle est bien supérieure à 0 et inférieure à la taille du fichier (- 17 octets).

5°) Donnez la portion de code permettant de lire les données situées à la position notée `pos`. Précisez les variables nécessaires éventuelles.

Solution : (2 pts)

```
int fd; /* Le descripteur du fichier à lire */
off_t posG, posD;
char c;
lseek(fd, pos, SEEK_SET);
read(fd, &posG, sizeof(off_t));
read(fd, &c, sizeof(char));
read(fd, &posD, sizeof(off_t));
```

6°) En sachant que les caractères de la phrase trouvée sont stockés dans le fichier binaire de sortie, est-il possible de voir le contenu de ce fichier avec un éditeur de texte classique ? Expliquez.

Solution : (1 pt) Un fichier texte correspond à une suite de caractères, ce qui est identique pour un fichier binaire ne contenant que des caractères. L'éditeur de texte est donc en mesure de le lire. Cependant, si un `\0` est ajouté à la fin de la chaîne, il pourrait être affiché dans l'éditeur.

Exercice 2 (Attrape-moi si tu peux - 7 pts)

Nous souhaitons écrire une application *ncurses* qui permet de visualiser la position de vaisseaux sur une grille de 40 par 20 (plusieurs vaisseaux peuvent se trouver sur la même case). Le programme principal est nommé *visualiseur* et les vaisseaux correspondent au programme nommé *vaisseau* qui peut être exécuté autant de fois que souhaité. Lorsque le vaisseau démarre, il génère une position aléatoire sur la grille puis l'envoie au *visualiseur* à l'aide d'un signal temps réel. Il se met en pause d'une seconde puis se déplace à une nouvelle position aléatoire. Il envoie cette dernière au *visualiseur*, toujours via un signal temps réel. L'utilisateur du *visualiseur* voit sur l'interface *ncurses* la position des vaisseaux représentée par un caractère 'X'. Une fenêtre d'informations affiche les créations des vaisseaux, leurs positions de départ et leurs déplacements. Le *visualiseur* s'arrête lorsque l'utilisateur presse `CRTL+C`.

1°) Comment le *visualiseur* peut-il identifier de manière unique chaque vaisseau pour n'afficher qu'un seul 'X' par vaisseau ?

Solution : (1 pt) Lorsque le vaisseau démarre, il envoie un signal au *visualiseur*. Celui-ci obtient donc son PID en plus de sa position. Il peut les placer dans une liste.

2°) Comment le vaisseau peut-il envoyer sa position au programme *main* avec un signal temps réel ? Comment peut-il le faire avec un seul signal ?

Solution : (1 pt) Lorsqu'on envoie un signal avec *sigqueue*, il est possible d'y associer une valeur de type *int*. Comme la position est constituée de 2 entiers sur un octet, on peut utiliser 2 octets sur les 4 de l'entier.

3°) Si le *visualiseur* s'arrête sans en avertir les vaisseaux, comment ces derniers peuvent-il le détecter ?

Solution : (1 pt) L'envoi du signal va tout simplement échouer. *sigqueue* retourne la valeur -1 et *errno* prendra une valeur spécifique.

4°) Donnez le code permettant au serveur d'attendre un signal d'un vaisseau.

Solution : (2 pts)

```

sigset_t ensemble;
siginfo_t info;

sigfillset(&ensemble);
sigprocmask(SIG_BLOCK, &ensemble, NULL);

sigemptyset(&ensemble);
sigaddset(&ensemble, SIGRTMIN + 1);
sigwaitinfo(&ensemble, &info);

```

Nous supposons l'existence de la structure `position_t` qui possède 2 champs `x` et `y`, de type `unsigned char`. Nous supposons également l'existence de la fonction `deplacement` qui calcule une position aléatoire et qui la retourne via une structure `position_t`.

5°) Donnez le code de la boucle principale d'un vaisseau.

Solution : (2 pts)

```

position_t position;
int valeur, fin = 0;
union sigval valeur;
pid_t pid = atoi(argv[1]);

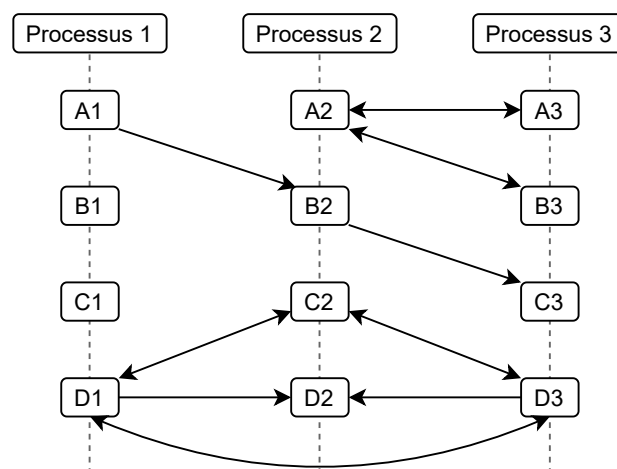
position = deplacement();
memcpy(&(valeur.sival_int), &position, sizeof(position_t));
sigqueue(pid, SIGRTMIN+1, valeur);

while(fin == 0) {
    sleep(1);
    position = deplacement();
    memcpy(&(valeur.sival_int), &position, sizeof(position_t));
    if(sigqueue(pid, SIGRTMIN+1, valeur) == -1)
        fin = 1;
}
printf("Le_visualiseur_est_mort,_vive_le_visualiseur_!\n");

```

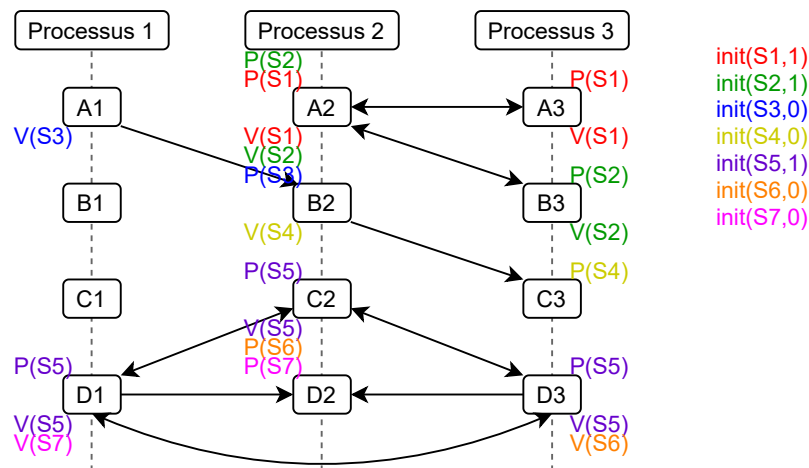
Exercice 3 (Sémaphores de Dijkstra et graphe d'allocation de ressources - 5 pts)

Soit le diagramme de précédence suivant, représentant 3 processus contenant chacun 4 blocs s'exécutant les uns après les autres :



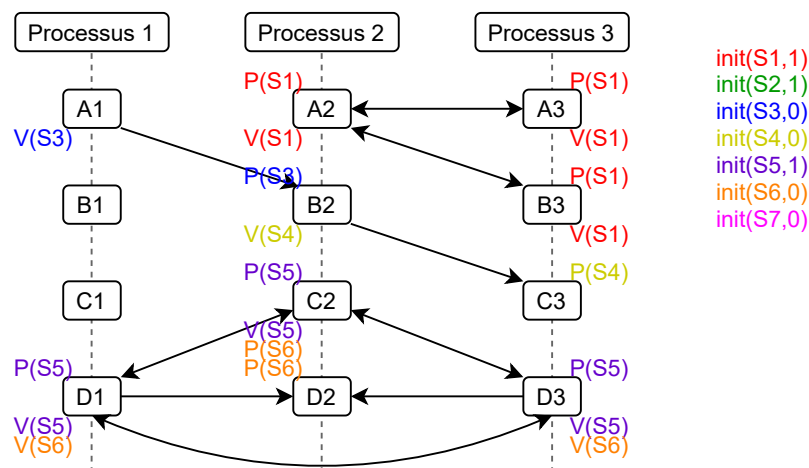
1°) Proposer une solution à base de sémaphores pour résoudre ces contraintes (1 sémaphore pour chaque cas).

Solution : (2pts) Voici une solution :



2°) Proposez une solution pour réduire le nombre de sémaphores.

Solution : (1pt) Le sémaphore 7 peut être supprimé en réutilisant le sémaphore 6. Les sémaphores 1 et 2 peuvent également être remplacés par un seul sémaphore. En effet, A3 est forcément exécuté avant B3 et ne peut pas s'exécuter en même temps. Donc quand A2 s'exécute, A3 et B3 ne peuvent pas s'exécuter en même temps.

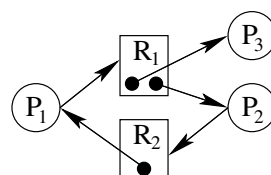


3°) Donnez un exemple de diagramme de précedence qui ne peut pas être résolu «correctement» par les sémaphores de Dijkstra et expliquez pourquoi.

Solution : (1 pt) Soit 3 sections A, B et C avec A et B en exclusion mutuelle et B et C également. Supposons que A entre en exclusion mutuelle. Comme le passage en exclusion mutuelle de B nécessite deux opérations différentes, il est possible qu'il bloque l'exécution de C alors que C pourrait s'exécuter en même temps que A.

4°) À l'aide d'un graphe d'allocation de ressources, montrez qu'un cycle n'induit pas forcément un interblocage. Vous expliquerez chaque élément du diagramme (en donnant un cas concret) et pourquoi il n'y a pas d'interblocage.

Solution : (1pt) Soit le diagramme suivant :



Supposons que $R1$ corresponde aux processeurs de la machine (2 exemplaires) et que $R2$ est un fichier qui ne doit être accessible que par un seul processus à la fois. On a bien un cycle et pourtant, quand $P3$ libèrera un exemplaire de $R1$, alors $P1$ pourra obtenir un exemplaire de $R1$. Une fois que $P1$ aura terminé, il libèrera l'exemplaire de $R2$ et $P2$ pourra l'obtenir à son tour.